# Plano final: Planeador de Viagens (Spring Boot + React)

Projeto full-stack para CV e LinkedIn, sem custos. O backend mantém-se completo, sem cortes, e o React substitui o Thymeleaf no frontend.

## Ferramentas (tudo grátis)

- JDK 21 (Temurin)
- IntelliJ IDEA Community
- Docker Desktop
- Git
- Node.js (LTS)
- VS Code (para o React)
- DBeaver (opcional)
- Conta no GitHub (repositório público)

As APIs externas (Nominatim, Open-Meteo, Frankfurter, OSRM) não precisam de registo nem de chave.

## Backend (completo, sem cortes)

### Fase 0: Setup (1 dia)

- Repositório no GitHub e projeto em start.spring.io (Web, JPA, Validation, PostgreSQL, Flyway, Security)
- `docker-compose.yml` com PostgreSQL
- Primeiro CI com GitHub Actions a correr `mvn test`

### Fase 1: Viagens e roteiro (1 a 2 semanas)

- Entidades `Viagem`, `Dia` e `Local`, com relações `@OneToMany`
- CRUD completo via REST e migrações Flyway
- Swagger/OpenAPI para testar tudo

### Fase 2: Mapa (1 semana)

- Nominatim para converter o nome do local em coordenadas
- Chamadas com `RestClient` e cache com Caffeine (respeitando 1 pedido por segundo)

### Fase 3: Orçamento (1 semana)

- Entidade `Despesa` (categoria, valor, moeda)
- Frankfurter para conversão de moedas, com cache
- Endpoint de resumo por categoria
- Testes unitários dos cálculos

### Fase 4: Utilizadores e segurança (1 a 2 semanas)

- Registo e login com Spring Security + JWT
- Cada utilizador só acede às suas viagens
- Link público de partilha (só leitura, com token aleatório)
- CORS configurado

### Fase 5: Extras técnicos

- Open-Meteo: previsão do tempo por dia
- OSRM: ordenar os locais de cada dia pela rota mais curta

### Fase 6: Qualidade (3 a 4 dias)

- `@ControllerAdvice` para erros consistentes
- Testes de integração com Testcontainers
- Dockerfile multi-stage do backend

## Aprender React (1 a 2 semanas, antes de tocar no frontend)

- Tutorial interativo em react.dev
- JavaScript/TypeScript básico, componentes, props, `useState` e `useEffect`
- Mini exercício (lista de tarefas) para ganhar confiança

## Frontend

**Stack:** React + TypeScript + Vite, React Router, TanStack Query, react-leaflet, Recharts e Tailwind CSS.

1. Login e registo (guardar o token)
2. Lista e criação de viagens
3. Roteiro dia a dia
4. Mapa com marcadores por dia

5. Orçamento com gráfico por categoria
6. Tempo previsto e ordenação por rota
7. Página pública de partilha

## Polimento final (1 semana)

- Docker Compose com 3 serviços (frontend com Nginx, backend, PostgreSQL)
- CI a testar o backend e a fazer build do frontend
- README com GIF da demo, screenshots e `docker compose up`
- Dados de demonstração e badge do CI
- Post no LinkedIn e linha no CV

## Tempo total

Cerca de **11 a 14 semanas** a ritmo calmo.

## Regras de ouro

- Só passas ao React quando a API estiver completa e testada no Swagger. Assim nunca depuras duas coisas novas ao mesmo tempo.
- Um branch por funcionalidade e Pull Request para a `main`.
- Commits pequenos e claros, quase todos os dias.
- Se bloqueares, escreve o que tentaste e o erro exato, para perceberes o problema sem receberes tudo feito.

## No CV/LinkedIn

*"Aplicação full-stack de planeamento de viagens: API REST em Spring Boot com JWT e PostgreSQL, frontend em React e TypeScript, integração com APIs externas (mapas, câmbios, meteorologia), testes com Testcontainers, containerizada com Docker e com CI em GitHub Actions."*